



FastAPI Interview Questions and Answers (Basic to Advanced)

Basic Level Questions

1. What is FastAPI? FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.6+ based on standard Python type hints. It is built on top of Starlette (for the web parts) and Pydantic (for data validation).

2. Why should you use FastAPI?

- Fast (high performance)
- Supports async and await
- Automatic data validation using Pydantic
- Auto-generated Swagger and ReDoc documentation
- Easy to use and intuitive

3. How do you define a simple GET endpoint in FastAPI?

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello World"}
```

4. What are Pydantic models in FastAPI? Pydantic models are used to define and validate the request and response schemas. They use Python type annotations to enforce data types.

5. How does FastAPI perform request validation? FastAPI uses Pydantic to validate the request body, query parameters, and path parameters based on defined data models.

6. What are the key features of FastAPI?

- Based on modern Python type hints
 - Built-in support for OAuth2, JWT
 - Dependency injection system
 - Data serialization and deserialization
 - Async support for concurrency
 - Interactive API docs via Swagger and ReDoc
-

Intermediate Level Questions

7. How do you handle path and query parameters in FastAPI?

```
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None):
    return {"item_id": item_id, "q": q}
```

8. How can you add request body in FastAPI?

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
def create_item(item: Item):
    return item
```

9. What is Dependency Injection in FastAPI? FastAPI provides a `Depends` mechanism to inject dependencies such as database sessions, authentication, etc.

```
from fastapi import Depends

def common_dependency():
    return "some data"

@app.get("/dependency/")
def read_dependency(data: str = Depends(common_dependency)):
    return {"data": data}
```

10. What is the use of BackgroundTasks in FastAPI? Allows you to run background operations after sending a response.

```
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message)

@app.post("/log")
def log_task(background_tasks: BackgroundTasks):
```

```
background_tasks.add_task(write_log, "log this message")
return {"message": "Task scheduled"}
```

11. How does FastAPI handle CORS? \ Using `CORSMiddleware` from `fastapi.middleware.cors`.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

12. How does FastAPI handle form data?

```
from fastapi import Form

@app.post("/submit/")
def submit_form(username: str = Form(...), password: str = Form(...)):
    return {"username": username, "password": password}
```

13. How do you upload files in FastAPI?

```
from fastapi import File, UploadFile

@app.post("/upload/")
def upload(file: UploadFile = File(...)):
    return {"filename": file.filename}
```

Advanced Level Questions

14. How do you integrate FastAPI with databases? \ Using libraries like SQLAlchemy, Tortoise ORM, or encode/databases. Typically you create a session dependency and use it in endpoints.

15. How do you implement JWT authentication in FastAPI?

- Use `pyjwt` or `python-jose` to encode/decode tokens
- Create login endpoint that issues JWT
- Protect endpoints with `Depends` and decode JWT to extract user info

16. What are event handlers in FastAPI? Used to execute startup and shutdown tasks.

```
@app.on_event("startup")
def on_startup():
    print("App started")

@app.on_event("shutdown")
def on_shutdown():
    print("App shutting down")
```

17. How does FastAPI generate API docs? FastAPI automatically generates Swagger UI and ReDoc documentation using OpenAPI.

- Swagger: `http://localhost:8000/docs`
- ReDoc: `http://localhost:8000/redoc`

18. How to handle exceptions globally in FastAPI?

```
from fastapi.responses import JSONResponse
from fastapi.requests import Request
from fastapi import HTTPException

@app.exception_handler(HTTPException)
def http_exception_handler(request: Request, exc: HTTPException):
    return JSONResponse(status_code=exc.status_code, content={"message":
exc.detail})
```

19. How do you create custom exception handlers in FastAPI?

```
class UnicornException(Exception):
    def __init__(self, name: str):
        self.name = name

@app.exception_handler(UnicornException)
def unicorn_exception_handler(request: Request, exc: UnicornException):
    return JSONResponse(status_code=418, content={"message": f"Oops! {exc.name}
did something wrong."})
```

20. How can you use OAuth2 with Password (and hashing), JWT tokens in FastAPI?

- Use `OAuth2PasswordBearer` to extract token
- Validate the token manually in a dependency
- Issue JWT in login endpoint
- Use `bcrypt` or `passlib` to hash and verify passwords

21. How can you serve static files in FastAPI?

```
from fastapi.staticfiles import StaticFiles

app.mount("/static", StaticFiles(directory="static"), name="static")
```

22. How can you run FastAPI with ASGI servers like Uvicorn or Hypercorn?

- Use command: `uvicorn main:app --reload`
- ASGI (Asynchronous Server Gateway Interface) allows for async concurrency in FastAPI apps

23. How do you write unit tests for FastAPI applications?

- Use `TestClient` from `fastapi.testclient`

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}
```

Let me know if you want this exported as a PDF or extended further.